Day 7 – Webhook Fundamentals (Synchronous)

🏷 Teacher Prep & Classroom Setup

1. **Pre-class Checklist**:
    - Ensure the virtual environment is active (`source .venv/bin/activate`).
    - Have the Django dev server stopped or running in the background.
    - Open a browser page to `https://webhook.site` to get a temporary test URL.
    - Open `games/webhooks.py` and `games/management/commands/check_expired_keys.py` in your IDE.
2. **Prerequisites Checklist**:
    - Students must have completed Day 6 successfully (Management command structured and working locally to expire keys).
3. **Required Material**:
    - Whiteboard or slide detailing the HTTP polling vs. webhook callback model, and the HMAC signing algorithm: `HMAC(webhook_secret, JSON_body) = Signature`

🎯 Learning Objectives

By the end of this class, students will be able to:

- Explain the concept of Webhooks (HTTP callbacks) and contrast them with polling.
- Implement cryptographic payload signing using HMAC-SHA256.
- Enforce deterministic JSON serialization using `sort_keys=True` in `json.dumps()`.
- Use the `requests` library to make synchronous POST requests with custom timeouts and headers.
- Identify the core limitations of synchronous HTTP calls in management commands.

⏱ Classroom Timeline (90 min)

Segment	Duration	Focus / Teacher Action
1. Hook & Big Picture	20 min	Explain the webhooks model. Contrast polling vs. event-driven callbacks.
2. Key Concepts & Core Ideas	20 min	Explain payload verification, HMAC, constant-time comparison, and JSON key sorting.
3. Live Coding Walkthrough	30 min	Write the synchronous webhook helper, integrate it with the management command, and test.
4. Discussion & Limitations	10 min	Analyze the failures of synchronous webhooks (blocking, timeout delays, no retry engine).

5. Check for Understanding	10 min	Q&A on security signatures, response codes, and sync blockages.

🚀 Lecture Step-by-Step Delivery Plan

1. Hook & Big Picture (20 min)

- **Teacher Action**: Contrast Polling vs Webhooks using a real-world analogy: *"Imagine checking your physical mailbox every 5 minutes to see if a package arrived (polling) versus having a delivery person ring your doorbell when it arrives (webhook). Webhooks are push notifications for APIs."*
- **Security Risk**: Explain that since any client can send a POST request to the publisher's public webhook URL, the publisher needs a way to guarantee the request actually came from our marketplace. We solve this using cryptographic payload signing.

2. Key Concepts & Core Ideas (20 min)

Introduce these webhook security and network concepts:

- **HMAC-SHA256**: Hash-based Message Authentication Code. Both systems share a secret. We sign the payload bytes with this secret, producing a signature header. The publisher recalculates the signature using the same secret and matches them.
- **`sort_keys=True` in JSON**: JSON objects are unordered maps. If keys are serialized in different orders on the sender and receiver, the raw byte sequences will differ, causing the HMAC signatures to mismatch. Sorting keys makes serialization deterministic.
- **`hmac.compare_digest()`**: A utility that compares digests in constant time. This prevents timing attacks (where attackers deduce correct signature bytes by measuring tiny variations in execution time).
- **Outbound Timeouts**: Why we must always set a timeout (e.g. `timeout=5`) when calling external services. Without a timeout, a hanging remote server will cause our command line process to block indefinitely.
- **`raise_for_status()`**: A method that raises an HTTP error if the server returns a `4xx` or `5xx` error code.

---

3. Live Coding Walkthrough (30 min)

Step 3.1: Implementing the Webhook Sender

- **Explain**: Create `games/webhooks.py`. We will write a helper function that constructs the expiry event payload, signs it using HMAC-SHA256, sets the custom headers, and dispatches the synchronous POST request.
- **Code**: Create `games/webhooks.py`:

```
import hmac
import hashlib
import json
import requests


def send_expiry_webhook(publisher, game_key_str, game_title, expire
```

```
    payload = {
        "event": "game_key.expired",
        "game_key": game_key_str,
        "game_title": game_title,
        "expired_at": expired_at.isoformat(),
    }

    secret = publisher.webhook_secret.encode()
    body = json.dumps(payload, sort_keys=True).encode()
    signature = hmac.new(secret, body, hashlib.sha256).hexdigest()

    headers = {
        "Content-Type": "application/json",
        "X-Signature": f"sha256={signature}",
    }

    try:
        response = requests.post(publisher.webhook_url, json=payloa
        response.raise_for_status()
    except Exception as e:
        print(f"[Webhook] Delivery failed for publisher {publisher.
```

- **Verify**: Check that the module imports compile without syntax errors.
- ⚠ **Common Pitfalls**:
  - **Type errors with secrets**: Explain that `hmac.new()` requires bytes, not strings. Both the `secret` and the `body` must be `.encode()`ed to bytes before hashing.

Step 3.2: Updating the Management Command

- **Explain**: Modify `check_expired_keys.py` to trigger the synchronous webhook sender for every key we mark as expired.
- **Code**: Update `games/management/commands/check_expired_keys.py`:

```
from django.core.management.base import BaseCommand
from django.utils import timezone
from games.models import GameKey
from games.webhooks import send_expiry_webhook


class Command(BaseCommand):
    help = 'Mark expired keys and notify publishers synchronously.'

    def handle(self, *args, **options):
        expired_keys = GameKey.objects.select_related('game__publis
            expires_at__lte=timezone.now(),
            status='active'
        )
        count = expired_keys.update(status='expired')

        for key in GameKey.objects.filter(status='expired').select_
            send_expiry_webhook(key.game.publisher, key.key_string,
```

```
self.stdout.write(f'Expired {count} keys (sync webhooks sen
```

- **Verify**:
  - Open `https://webhook.site` in a browser and copy the unique URL.
  - Create a publisher in your Django admin panel and paste this URL into their `webhook_url` field.
  - Expire a key in admin (by setting `expires_at` in the past and status to `active`).
  - Run `python manage.py check_expired_keys`.
  - Check the `webhook.site` logs to confirm that the HTTP POST request arrived with the correct JSON body and the `X-Signature` header.

---

4. Discussion & Limitations (10 min)

- **Teacher Action**: Open a discussion on the problems with this synchronous approach:
  1. **Thread Blocking**: The entire script blocks during the HTTP call.
  2. **Cascading Slowness**: If one publisher's server is slow, the script takes longer, delaying webhooks for all subsequent publishers.
  3. **No Resiliency (Retries)**: If a publisher's server is down (connection error), the webhook is lost forever.

  - *Preview: Explain that tomorrow we will solve these three issues using an asynchronous task queue (Celery + Redis).*

---

5. Check for Understanding (10 min)

Question: "A publisher claims they received a webhook from us but the payload was tampered with. How does HMAC protect against this?"

Answer: HMAC uses a shared secret to generate a cryptographic signature of the request body. If any character in the payload is altered during transit, the receiver's computed signature will not match the `X-Signature` header, revealing that the payload was modified and should be rejected.

Question: "Why do we pass `sort_keys=True` to `json.dumps()`? What could go wrong without it?"

Answer: JSON keys are unordered. If we don't sort keys, the serialization format might change (e.g. dictionary keys serialized in a different order), resulting in a different byte sequence. This would produce a different HMAC signature, causing verification to fail even if the content remains identical.

Question: "What happens to the management command if the publisher's server is down and we're making synchronous HTTP calls?"

Answer: The management command thread will block, waiting for the HTTP request to timeout (e.g. 5 seconds per request). If many keys expired for that publisher, or multiple publishers are down, the script will take a very long time to complete and could cause other pending updates to stall.

Question: "What's the difference between a 4xx and 5xx response from the

publisher's webhook endpoint? Should we retry both?"

Answer: A `4xx` response represents a client error (e.g., `400 Bad Request` or `404 Not Found`), suggesting our payload is invalid or the endpoint is misconfigured; retrying will likely fail again, so we shouldn't retry. A `5xx` response represents a server error (e.g., `500 Internal Server Error` or `503 Service Unavailable`), suggesting temporary publisher outage; we should retry these as the server might recover.

---

6. Daily Checkpoint & Wrap-up (5 min)

- **Verification Criteria**:
  - ○ The `send_expiry_webhook` function is implemented in `games/webhooks.py`.
  - ○ Running `python manage.py check_expired_keys` dispatches an HTTP POST request to the publisher's webhook URL.
  - ○ The HTTP request header contains a valid `X-Signature` starting with `sha256=`.
  - ○ The webhook payload arrives at `webhook.site` successfully.